School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

Fall Semester 2026-27

ASSESSMENT VIII – PROJECT REVIEW 1

DagOpt: DAG-Based Optimization of Basic Blocks

Common Subexpression and Redundant Computation Elimination using Directed Acyclic Graphs

Programme & Branch	B.Tech (CSE & Specialization) & SCOPE
Course Name & Code	Compiler Design Lab (BCSE307P)
Class Number	VL2026270103361
Slot	L15+L16
Faculty Name	Dr. Bhawana Tyagi
Review Date	07/10/2026

Team Members

S.No	Name	Register Number
1	Harshit Pandey	24BCT0173
2	Ankush Kumar	24BCT0254
3	Lakshya Kulshrestha	24BCT0299

1. Problem Identification and Objectives

1.1 Problem Statement

Compilers translate source programs into intermediate code, usually three-address code (TAC). Code produced directly from the front end is rarely efficient: the same expression is often computed several times, and values are recomputed even when nothing they depend on has changed. These redundant computations waste execution time and, at the machine level, registers and memory traffic.

Our assigned topic is: **implement a DAG-based optimization technique for basic blocks. Construct the DAG representation of a basic block and use it to eliminate common subexpressions and redundant computations.**

For example, in the block below the expression a + b is evaluated three times (once as b + a), although a and b never change:

```
t1 = a + b
t2 = a + b
t3 = b + a
```

1.2 Why a DAG?

A basic block is a straight-line sequence of instructions with one entry and one exit. Representing it as a directed acyclic graph captures how values flow through the block. Each distinct value is a single node, and every variable that holds that value is attached to the node as a label. When a new instruction would compute a value that already has a node, no new node is made. The existing node is reused, which is exactly the elimination of a common subexpression. A DAG therefore finds redundancy in a single pass over the block, without a separate data-flow analysis.

1.3 Objectives

1. Design a TAC input format and a parser that reads it into an internal instruction representation.
2. Partition the TAC into basic blocks using the standard leader rules.
3. Construct a DAG for each basic block, with value-sharing of identical operations.
4. Detect and eliminate common subexpressions, including those written in a different operand order for commutative operators (a + b and b + a).
5. Produce optimized intermediate code from the DAG and compare it with the original code (instruction counts before and after).
6. Validate that the optimization preserves program meaning, with unit tests and equivalence checking.

1.4 Scope

This report covers the foundation of the tool: the TAC front end (parsing), basic block construction, and the core DAG construction with common subexpression detection (objectives 1 to 4). Code generation from the DAG, extended simplifications, verification and result analysis (objectives 5 and 6) follow in the later stages of the project.

2. Literature Survey

Code optimization on basic blocks has a long history. The works below shaped our approach.

No.	Work	Contribution	Relevance to this project
1	Aho, Lam, Sethi & Ullman, Compilers: Principles, Techniques, and Tools (2nd ed., 2006), Ch. 8–9	Defines basic blocks, leaders and flow graphs, the DAG representation of basic blocks, and the DAG construction algorithm	Primary reference for our block splitting and DAG algorithm
2	Cocke & Schwartz, Programming Languages and Their Compilers (1970)	Early work on common subexpression elimination, value numbering and local optimization	Origin of the idea of assigning one identity to each computed value
3	Aho & Ullman, "Optimization of Straight Line Code", SIAM J. Computing, 1972	Theory of optimizing straight-line code using DAGs	Justifies DAGs as the right structure for a basic block
4	Aho, Johnson & Ullman, "Code Generation for Expressions with Common Subexpressions", JACM, 1977	Shows how to generate code from DAGs that contain shared nodes	Guides turning the optimized DAG back into code
5	Briggs, Cooper & Simpson, "Value Numbering", Software: Practice & Experience, 1997	Practical hash-based value numbering and its extension beyond a single block	Hash-table lookup of (operator, operands) is the same idea as our DAG node sharing
6	Cooper & Torczon, Engineering a Compiler (2nd ed., 2011)	Local value numbering, algebraic identities and redundancy elimination	Source of the algebraic and commutativity rules we use
7	Muchnick, Advanced Compiler Design and Implementation (1997)	Survey of machine-independent optimizations, including CSE and constant folding	Background for the optimizations the tool supports
8	Lattner & Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation", CGO, 2004	Modern compiler infrastructure with passes such as early CSE and instruction combining	Shows how block-level CSE is used in a production compiler

2.1 Observations

- DAG-based local optimization and local value numbering are two views of the same technique. Both give each computed value a unique identity and reuse it.
- Textbook treatments describe the algorithm but rarely give a runnable tool that shows the DAG and the effect of optimization.

- Production compilers hide these steps inside large frameworks, which makes them hard to study in isolation.

2.2 Gap and Our Contribution

We provide a small, self-contained command-line tool that follows the textbook DAG algorithm, keeps each stage separate (parse, split, build DAG), and can be tested stage by stage. This makes the optimization easy to inspect and explain.

3. Proposed Approach

The tool is a pipeline. Each stage takes the output of the previous stage. The table below shows the stages and their status.

Stage	Name	Purpose	Status
1	TAC parsing	Read the .tac file and build a list of instructions	Completed
2	Basic block splitting	Divide the instruction list into basic blocks at leaders	Completed
3	DAG construction	Build one DAG per block, sharing identical computations (CSE)	Completed
4	Optimized code output	Regenerate TAC from the DAG	Planned
5	Verification and analysis	Check correctness and report results	Planned

Implementation language: Python 3.10+. The project is organised as a package (dagopt) with one module per stage, so each stage can be unit tested on its own.

4. Implementation

4.1 Module Summary

Module	Responsibility	Related tests
tac.py	TAC data structures, parser and formatter; reports syntax errors	test_tac.py
blocks.py	Leader detection and basic block splitting (build_blocks)	test_blocks.py
dag.py	DAG node structure and construction; value sharing for common subexpressions; commutative canonicalization	test_dag.py

4.2 Input Format: Three-Address Code

Each non-blank, non-comment line of the input file is one of the forms below. Operands are variable names matching [A-Za-z_][A-Za-z0-9_]* or integer literals. Text after # is a comment.

Form	Example	Meaning
x = y	a = b	Copy or integer literal assignment
x = - y	a = - b	Unary negation

Form	Example	Meaning
x = y OP z	c = a + b	Binary operation (+ - * / %)
L:	loop:	Label declaration
goto L	goto loop	Unconditional branch
if a REL b goto L	if x < 10 goto loop	Conditional branch (< <= > >= == !=)

4.3 Basic Block Splitting

A leader is the first instruction of a basic block. The splitter marks leaders by three rules and then cuts the instruction list before each leader.

1. The first instruction of the program is a leader.
2. Any instruction that is the target of a jump (a label) is a leader.
3. Any instruction that immediately follows a jump is a leader.

Example input and the blocks produced:

```
i = 0
loop:
t1 = a + b
i = i + 1
if i < 10 goto loop
x = t1
```

Block	Instructions	Leader reason
B0	i = 0	First instruction
B1	loop: t1 = a + b; i = i + 1; if i < 10 goto loop	Label is a jump target
B2	x = t1	Follows a conditional jump

4.4 DAG Construction Algorithm

For each block, the DAG is built by processing instructions in order. A table maps each variable to the node that currently holds its value, and a hash table maps each (operator, left node, right node) key to an existing node.

```
for each instruction x = y OP z in the block:
    ny = node_of(y)      # create a leaf if y has no node yet
    nz = node_of(z)
    if OP is commutative: order (ny, nz) canonically
    key = (OP, ny, nz)
    if key exists in hash table:
        n = hash[key]    # common subexpression: reuse node
    else:
        n = new node(OP, ny, nz); hash[key] = n
    detach x from its old node; attach x to n as a label
```

The key points are the hash lookup, which removes the repeated computation, and the canonical ordering of operands for commutative operators (+ and *), which makes a + b and b + a map to the same key. Non-commutative operators such as - and / keep their operand order.

Copy instructions (x = y) simply attach x to the node of y, and unary operations use a key of the form (OP, n). Because keys are built from node identities and not from variable names, a reassigned variable automatically gets a different node. For example, in t1 = a + b; a = 5; t2 = a + b the second sum is not shared with the first, since a now refers to a new leaf. This is what keeps the optimization safe.

4.5 Worked Example

Input block:

```
t1 = a + b
t2 = a + b
t3 = b + a
x  = t1 * c
y  = t2 * c
z  = t3 + x
```

Resulting DAG nodes:

Node	Operator	Children	Labels	Note
n1	+	a, b	t1, t2, t3	t2 and t3 reuse n1 (t3 by commutativity)
n2	*	n1, c	x, y	y reuses n2
n3	+	n1, n2	z	New computation

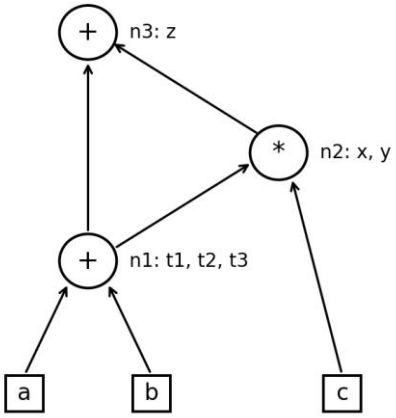


Figure 1: DAG of the worked example (circles are operations, boxes are leaves)

Six instructions contain six arithmetic operations, but the DAG needs only three operation nodes (n1, n2, n3). Three computations are therefore redundant and are removed once code is regenerated from the DAG.

References

1. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson/Addison-Wesley, 2006.

2. J. Cocke and J. T. Schwartz, Programming Languages and Their Compilers: Preliminary Notes. Courant Institute, New York University, 1970.
3. A. V. Aho and J. D. Ullman, "Optimization of straight line code," SIAM Journal on Computing, vol. 1, no. 1, pp. 1–19, 1972.
4. A. V. Aho, S. C. Johnson and J. D. Ullman, "Code generation for expressions with common subexpressions," Journal of the ACM, vol. 24, no. 1, pp. 146–160, 1977.
5. P. Briggs, K. D. Cooper and L. T. Simpson, "Value numbering," Software: Practice and Experience, vol. 27, no. 6, pp. 701–724, 1997.
6. K. D. Cooper and L. Torczon, Engineering a Compiler, 2nd ed. Morgan Kaufmann, 2011.
7. S. S. Muchnick, Advanced Compiler Design and Implementation. Morgan Kaufmann, 1997.
8. C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in Proc. IEEE/ACM CGO, 2004, pp. 75–86.